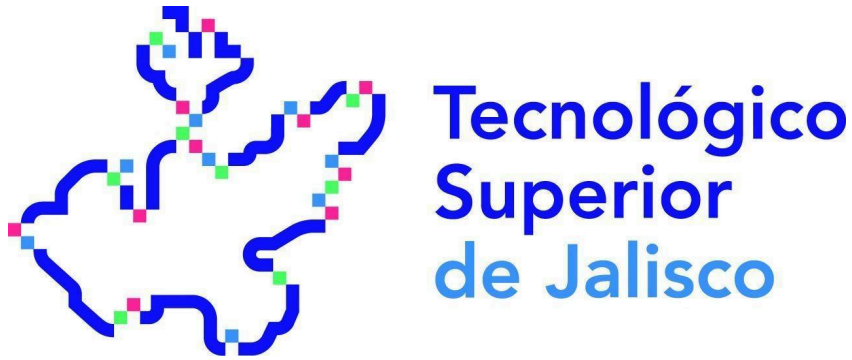




TECNOLÓGICO SUPERIOR DE JALISCO

UNIDAD ACADÉMICA LA HUERTA

ING. SISTEMAS COMPUTACIONALES

MATERIA: DEVOPS

ACTIVIDAD: PROYECTO INTEGRADOR

ALUMNO: ANGÉLICA GETSEMANÍ MUÑOZ JIMENEZ

MAESTRO: JOSE LUIS GONZALEZ

21/MAYO/2026

Reporte Ejecutivo Integral: Implementación de DevSecOps en API de Inventario Agrícola

1. Resumen Ejecutivo

El presente documento detalla la planificación, diseño e implementación de un pipeline de integración y despliegue continuo (CI/CD) bajo la filosofía DevSecOps. El proyecto se aplicó sobre una Interfaz de Programación de Aplicaciones (API) desarrollada en Node.js, diseñada para la gestión de inventarios agrícolas (lotes de limón persa). El objetivo principal fue automatizar la detección y mitigación de vulnerabilidades desde las fases tempranas del desarrollo ("Shift-Left Security"), garantizando que ningún código inseguro alcance los entornos de producción. Se logró la integración exitosa de pruebas de seguridad estática (SAST), análisis de composición de software (SCA) y escaneo dinámico de contenedores, culminando en un pipeline robusto capaz de bloquear despliegues ante la presencia de vulnerabilidades críticas o altas.

2. Contexto y Lógica de Negocio del Proyecto

Para dotar al proyecto de una lógica de negocio real y funcional, se desarrolló una API encargada de gestionar el inventario de una empacadora agrícola. El sistema permite registrar y consultar lotes de limón persa, clasificándolos por calidad (ej. Exportación, Segunda) y volumen. Esta arquitectura basada en microservicios ligeros es ideal para su contenedorización mediante Docker, lo que a su vez proporciona el escenario perfecto para aplicar prácticas avanzadas de seguridad en infraestructuras inmutables.

La elección de este modelo de negocio responde a la necesidad de proteger sistemas de trazabilidad agrícola, donde la integridad de los datos de inventario es crítica para las operaciones logísticas y comerciales.

3. Fundamentos de Arquitectura DevSecOps

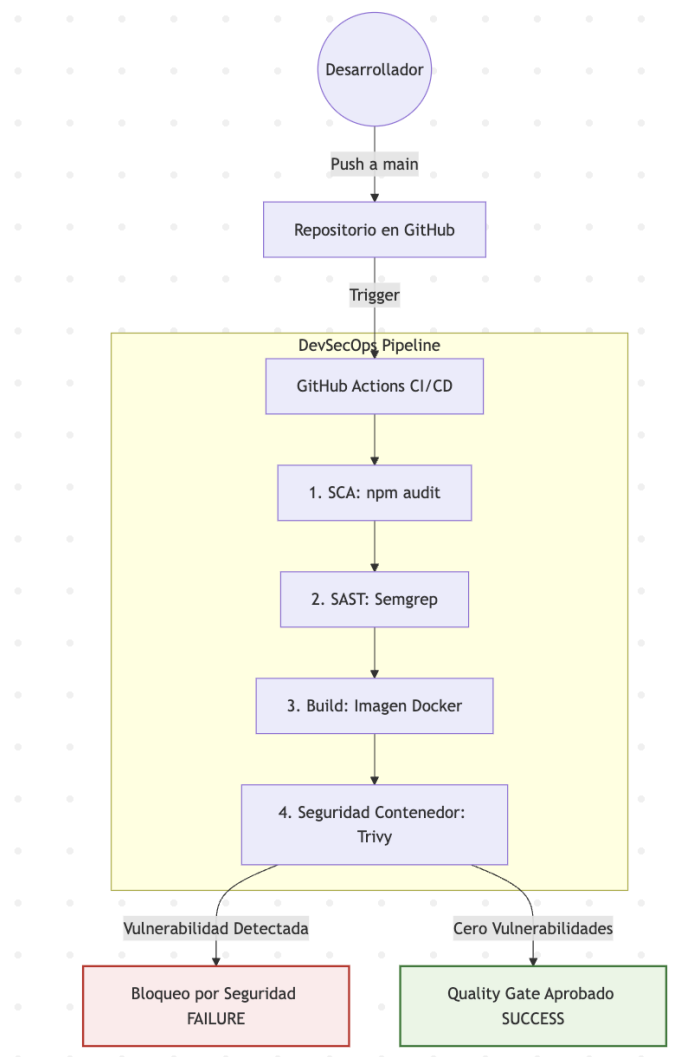
La arquitectura del pipeline se construyó utilizando **GitHub Actions** como orquestador principal. La decisión de utilizar este orquestador radica en su integración nativa con el repositorio de código fuente, reduciendo la fricción operacional y permitiendo una ejecución

fluida de los *workflows* basados en eventos (Push / Pull Requests).

El pipeline fue diseñado bajo el principio de *"Fail Fast, Fail Secure"* (Falla rápido, falla seguro), lo que significa que el proceso se interrumpe de manera inmediata (status: FAILURE) si cualquiera de las compuertas de seguridad (Security Gates) detecta una anomalía que supere el umbral de riesgo permitido (Severidad: Alta o Crítica).

3.1 Diagrama de Arquitectura del Pipeline

El siguiente diagrama ilustra el flujo de los eventos automatizados, desde la confirmación del código hasta la validación de la imagen del contenedor.



4. Herramientas y Controles de Seguridad Implementados

Para cumplir con los estándares de la industria, se integraron tres capas de seguridad

defensiva dentro del pipeline:

4.1 Análisis de Composición de Software (SCA) - npm audit

Las aplicaciones modernas están compuestas en más de un 70% por código de terceros (librerías open source). Se implementó npm audit configurado en un nivel de auditoría alto (--audit-level=high). Esta herramienta cruza las versiones de las dependencias declaradas en el proyecto contra bases de datos de vulnerabilidades conocidas (CVEs), bloqueando la construcción si el árbol de dependencias incluye paquetes comprometidos.

4.2 Pruebas de Seguridad Estática de Aplicaciones (SAST) - Semgrep

A diferencia de SCA, SAST analiza el código escrito por los desarrolladores y los archivos de configuración en busca de malas prácticas. Se configuró **Semgrep** mediante una imagen efímera de Docker para escanear tanto el código fuente de Node.js como las directivas del Dockerfile. Semgrep es altamente valorado en DevSecOps por su rapidez y su capacidad para detectar inyecciones, configuraciones erróneas y fallas lógicas sin necesidad de compilar el código.

4.3 Seguridad de Contenedores y SO - Trivy (Aqua Security)

Una vez que el código y las dependencias son validadas, el pipeline construye la imagen de Docker. Sin embargo, el sistema operativo base del contenedor (Alpine Linux) también representa un vector de ataque. Se integró **Trivy** a través del aquasecurity/trivy-action para realizar un escaneo profundo de la imagen resultante (api-limones:latest). El umbral de aceptación se configuró de forma estricta: el proceso retorna un exit-code: 1 (Bloqueo) ante vulnerabilidades marcadas como CRITICAL o HIGH en paquetes del sistema operativo o librerías.

5. Hallazgos de Seguridad y Mitigaciones (Proceso Iterativo)

La implementación del pipeline demostró su eficacia al bloquear repetidamente el despliegue debido a vulnerabilidades detectadas en tiempo real. A continuación se

documentan los incidentes críticos resueltos durante el proyecto:

Capa de Seguridad	Vulnerabilidad Detectada	Nivel de Riesgo	Estrategia de Mitigación Implementada
SAST (Semgrep)	Ejecución de contenedor con privilegios de Root (dockerfile.security.missing-user)	Crítico / Bloqueo	Se modificó el Dockerfile aplicando el principio de mínimo privilegio. Se inyectó la directiva USER node antes del comando de ejecución para asegurar que el proceso principal del contenedor corra sin permisos administrativos.
Seguridad de Contenedor (Trivy)	Vulnerabilidades múltiples en el SO base debido al uso de una imagen desactualizada (node:18-alpine).	Alto / Crítico	Actualización de la imagen base a la versión más reciente con soporte activo: FROM node:22-alpine, garantizando parches de seguridad del sistema operativo a nivel de kernel.
Seguridad de Contenedor (Trivy)	Vulnerabilidades en el gestor de paquetes incluido de fábrica en	Alto	Se introdujo un paso de limpieza en la construcción del

Capa de Seguridad	Vulnerabilidad Detectada	Nivel de Riesgo	Estrategia de Mitigación Implementada
	la imagen de Node.		contenedor mediante la directiva RUN npm install -g npm@latest, forzando la actualización de las herramientas internas antes de construir la aplicación.
SCA (npm audit / Trivy)	Vulnerabilidades profundas en dependencias transitivas (ej. cross-spawn, tar, glob).	Alto	Uso avanzado del archivo package.json implementando la propiedad "overrides". Esto forzó la resolución del árbol de dependencias hacia versiones parcheadas (ej. tar a la versión 7.5.11), eliminando la superficie de ataque heredada.

6. Evidencias Gráficas del Pipeline DevSecOps

La validación del modelo se documenta mediante las siguientes capturas del sistema de integración continua, demostrando el comportamiento de bloqueo y la posterior aprobación tras la remediación.

6.1 Simulación de Interrupción de Despliegue (Bloqueo por Seguridad)

En esta captura se evidencia cómo el *Quality Gate* se cierra, impidiendo que código inseguro o contenedores con vulnerabilidades conocidas avancen en el ciclo de vida.

 Cambio para video DevSecOps Pipeline #6: Commit 5e98454 pushed by getseMunoz	main	 30 minutes ago  44s	...
 Fix: Fuerza el uso de versiones seguras en las dependencias pa... DevSecOps Pipeline #4: Commit 64fb087 pushed by getseMunoz	main	 42 minutes ago  56s	...

6.2 Despliegue Exitoso Post-Mitigación (Pipeline Aprobado)

Tras aplicar las correcciones detalladas en la sección 5, el código es validado exitosamente en todos sus controles de seguridad, resultando en un artefacto confiable y listo para su distribución.

 Arreglo para video DevSecOps Pipeline #7: Commit 50b7f2a pushed by getseMunoz	main	 28 minutes ago  1m 16s	...
 Fix: Actualiza imagen a Node 22 y fuerza actualizacion de npm ... DevSecOps Pipeline #5: Commit 43dc403 pushed by getseMunoz	main	 38 minutes ago  1m 5s	...

7. Estrategia de Gestión de Secretos

El manejo inadecuado de credenciales representa una de las vulnerabilidades más explotadas en la industria (OWASP Top 10). Para este proyecto, se adoptó un enfoque preventivo integral respecto a la gestión de secretos:

1. **Filtros de Control de Versiones:** Se implementó un archivo `.gitignore` desde el inicio (commit 0) del repositorio. Este archivo fue configurado explícitamente para ignorar el directorio `node_modules/` y cualquier archivo de variables de entorno (`.env`).
2. **Revisión de Historial:** Se auditó el repositorio para confirmar que no existen *commits* históricos que contengan cadenas de conexión a bases de datos, claves de API o tokens de autorización codificados en texto plano (*hardcoded*).
3. **Inyección Dinámica:** La aplicación está diseñada para consumir variables de configuración a través del objeto `process.env` de Node.js, lo que permite que en un entorno de producción, las credenciales sean inyectadas de forma segura por un Vault o

directamente desde los *Secrets* del orquestador.

8. Conclusiones y Lecciones Aprendidas

El desarrollo de este proyecto integrador evidenció que la seguridad informática no es un paso final aplicable antes de la entrega, sino una cultura que debe integrarse transversalmente en el desarrollo. Entre las principales lecciones aprendidas destacan:

- **El mito de las imágenes base "seguras":** Se comprobó empíricamente que utilizar imágenes oficiales (como `node:alpine`) no exime de responsabilidades de seguridad. Las imágenes de contenedores envejecen rápidamente, y dependencias internas de fábrica pueden albergar vulnerabilidades críticas que requieren mitigación explícita (como la actualización forzada de `npm`).
- **El riesgo de las dependencias transitivas:** La corrección de vulnerabilidades no siempre es tan directa como actualizar un paquete. A través del uso de la función `overrides`, se comprendió el desafío de asegurar librerías de terceros sobre las cuales el desarrollador no tiene control directo, y cómo herramientas como Trivy son vitales para mapear este árbol oculto.
- **Automatización como pilar operativo:** Implementar SAST y SCA de forma manual resulta inescalable. La configuración en GitHub Actions demostró cómo las herramientas automatizadas actúan como guardianes eficientes, liberando al desarrollador para concentrarse en la lógica de negocio mientras el pipeline asegura la calidad del entregable.

La adopción de DevSecOps en este caso práctico no solo protegió a la aplicación de vectores de ataque como la escalación de privilegios o envenenamiento de paquetes, sino que estableció una línea base de calidad de software altamente profesional.